Torre de Hanói

Universidade Federal de Roraima (UFRR)
Disciplina: Análise de Algoritmos
Professor: Prof. Dr. Herbert Oliveira Rocha
Equipe:

- Helian Silva
- José Neto

Objetivo:

Analisar o comportamento computacional do algoritmo da **Torre de Hanói,** comparando duas implementações:

1. **Recursiva**
2. **Iterativa**

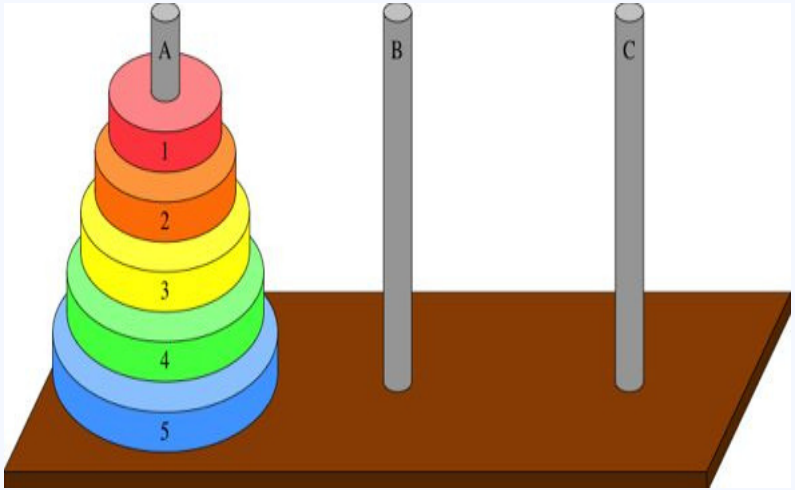
O foco do trabalho é observar:

- função de custo;
- complexidade assintótica;
- tempo de execução;
- diferenças práticas entre as implementações.

O que é a Torre de Hanói?

A Torre de Hanói é um problema clássico da matemática e da computação, muito utilizado no estudo de recursão e análise de algoritmos.

O problema consiste em mover uma pilha de discos de uma haste de origem para uma haste de destino, utilizando uma haste auxiliar, respeitando regras específicas.



Regras:

- Apenas um disco pode ser movido por vez;
- Um disco maior nunca pode ser colocado sobre um disco menor;
- Todos os discos começam empilhados na haste de origem em ordem decrescente de tamanho.

Função de custo da Torre de Hanói

Recorrência

A função de custo da Torre de Hanói é: $T(n) = T(n-1) + 1 + T(n-1)$

Por que essa fórmula?

Porque, para resolver o problema com n discos, o algoritmo:

- n−1: Movimentos para deslocar o bloco superior para o pino auxiliar para liberar a base.
- (+1): Movimento para deslocar o maior disco diretamente para o pino de destino.
- n−1 discos: Movimentos para deslocar o bloco do pino auxiliar para cima da base no destino.

Dedução da Recorrência

Expandindo a fórmula: $T(n) = T(n-1) + 1 + T(n-1)$

Temos: $T(n) = 2T(n-1) + 1$

$$T(n) = 2(2T(n-2) + 1) + 1$$

$$T(n) = 2^2T(n-2) + 2 + 1$$

$$T(n) = 2^2(2T(n-3) + 1) + 2 + 1$$

$$T(n) = 2^3T(n-3) + 2^2 + 2^1 + 2^0$$

$$T(n) = 2^iT(n-i) + \sum_{j=0}^{i-1} 2^j$$

$$T(n) = 2^nT(0) + \sum_{j=0}^{n-1} 2^j$$

$$S = \frac{1(2^n - 1)}{2 - 1} = 2^n - 1$$

Resultado: $S = 2^n - 1$

Prova por indução

Queremos provar $\mathbf{J(n) = 2^n - 1}$ sabendo que: $T(n) = 2T(n-1) + 1$

Base: n = 1

$$J(1) = 2^1 - 1 = 1 = T(1)$$
$$J(2) = 2^2 - 1 = 3 = T(2)$$

Queremos Provar: $T(n) = J(n) = 2^n - 1$

Supondo que : $T(n) = 2.T(n-1) + 1$ é sempre verdade:

$$T(n) = 2.T(n-1) + 1$$
$$T(n) = 2.J(n-1) + 1$$
$$T(n) = 2.(2^{n-1} - 1) + 1$$
$$T(n) = 2.2^{n-1} - 2 + 1$$
$$T(n) = 2^1.2^{n-1} - 1$$

$$T(n) = 2^n - 1$$

Complexidade assintótica da Torre de Hanói

- A solução da recorrência é: $T(n) = 2^n - 1$
- O termo que domina o crescimento é 2^n Logo, a complexidade assintótica é: $O(2^n)$
- Segundo Cormen, na Torre de Hanói o melhor e o pior caso são idênticos, pois o algoritmo é determinístico e o número de movimentos depende apenas do número de discos n.

Algoritmo Recursivo

- **Auto-referente:** resolve o problema chamando a si mesmo para tratar partes menores.
- **Ramificado:** divide o desafio em duas etapas menores antes e depois de cada movimento.
- **Ineficiente:** precisa guardar cada etapa pendente até que os discos menores sejam movidos.

```
Algoritmo TorreHanoiRecursiva(n, origem, destino, auxiliar)

    se n = 0 então
        retornar
    fim_se

    TorreHanoiRecursiva(n – 1, origem, auxiliar, destino)

    mover um disco da origem para o destino

    TorreHanoiRecursiva(n – 1, auxiliar, destino, origem)

fim_algoritmo
```

Algoritmo Iterativo

- **Direto:** percorre os movimentos em sequência, sem voltar atrás nem guardar passos antigos.
- **Eficiente:** usa pouco espaço de memória, sempre a mesma quantidade do início ao fim.
- **Rápido:** utiliza cálculos matemáticos muito simples.

```
Algoritmo TorreHanoiIterativa(n)

    total_movimentos ← (2^n) - 1

    para i ← 1 até total_movimentos faça
        origem  ← (i AND (i - 1)) mod 3
        destino ← ((i OR (i - 1)) + 1) mod

        mover disco da origem para o desti
    fim_para

fim_algoritmo
```

Metodologia dos Testes

Os testes foram realizados em linguagem C, utilizando repetição automática para medir o tempo médio de execução.

Configuração

linguagem: **C**
ambiente: **Windows**
medição de tempo: **microssegundos**
repetições por teste: **20**

Entradas testadas

n = 10
n = 15
n = 20
n = 25
n = 30
n = 35

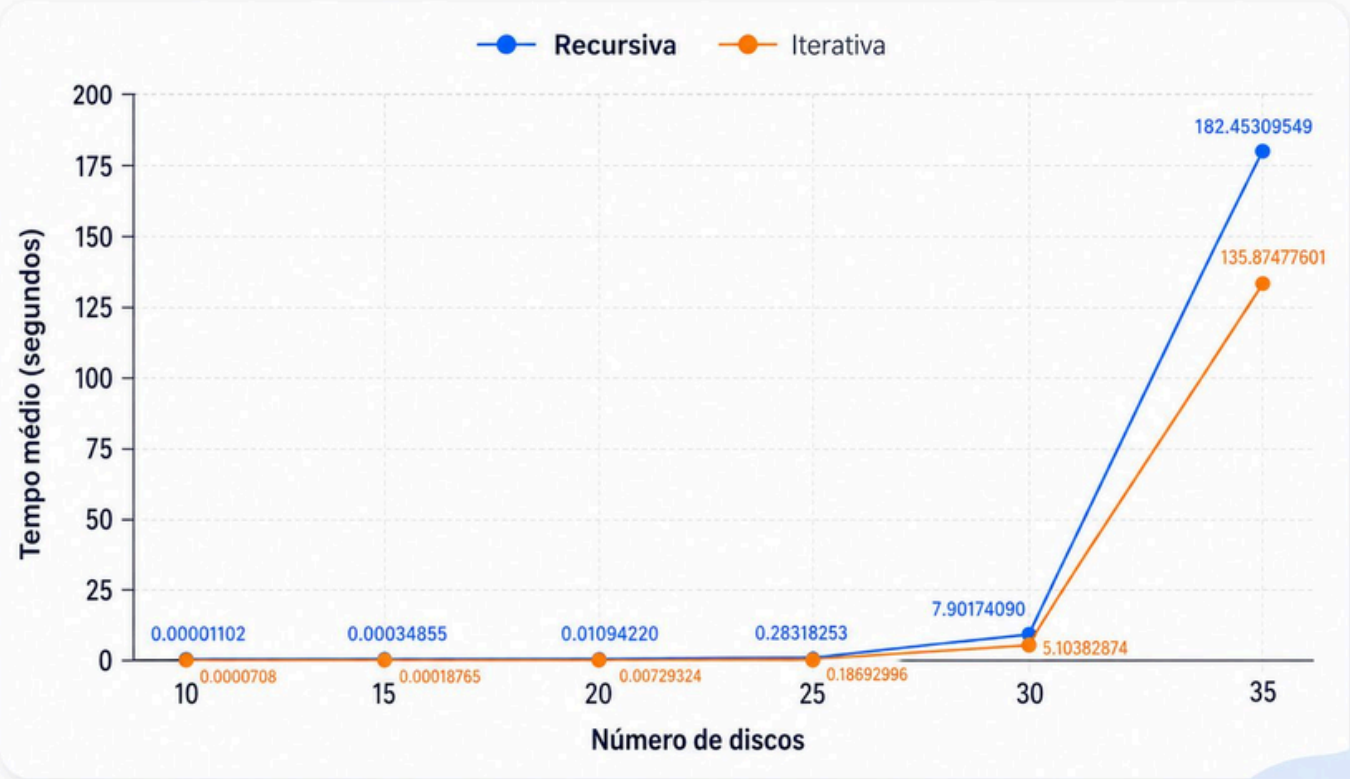
Considerações sobre o ambiente

Durante os testes, pequenas variações de tempo podem ocorrer por fatores do ambiente de execução, como:

- escalonamento do sistema operacional;
- uso dos níveis de cache da CPU;
- aquecimento do processador em execuções longas.

Comparação entre Torre de Hanói Recursiva e Iterativa

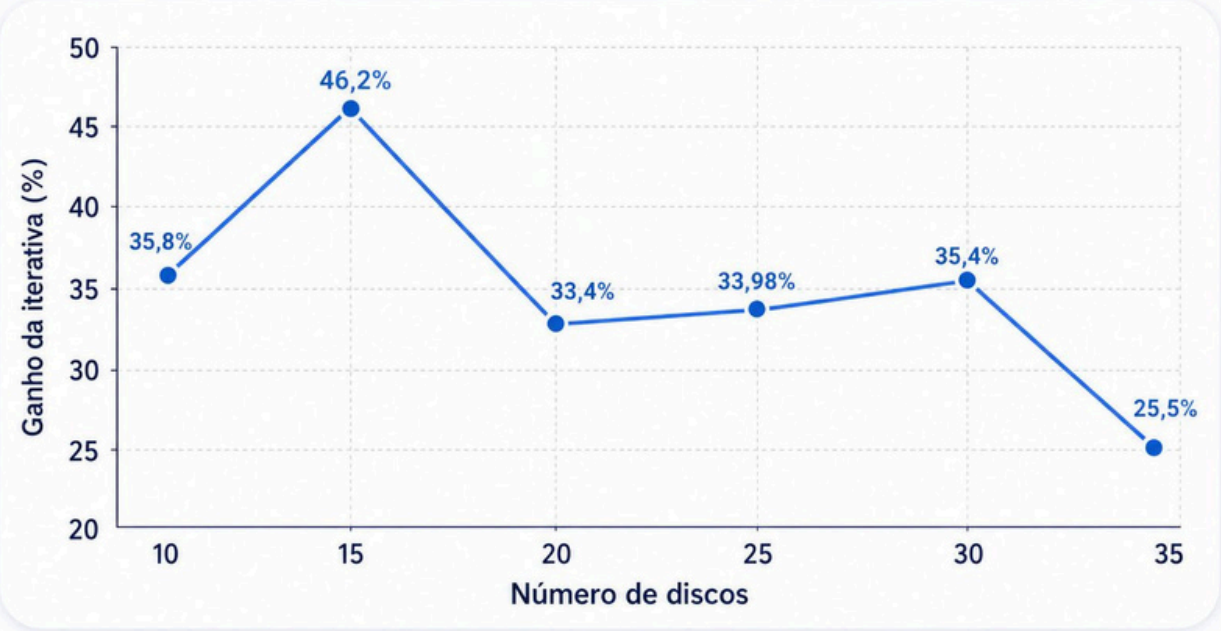
Tempo médio de execução por número de discos



Fonte: testes experimentais da equipe

Ganho da Versão Iterativa

Percentual de melhoria em relação à versão recursiva



A versão iterativa foi mais rápida em todos os testes.

Fonte: testes experimentais da equipe

Mesmo que a complexidade teórica seja a mesma, a forma como o algoritmo é implementado influencia diretamente o desempenho observado no gráfico.

Na versão recursiva:

- Divide o problema em várias partes menores;
- Precisa organizar essas partes antes de continuar;

Essa organização extra aumenta o tempo de execução.

Na versão iterativa:

- Executa os movimentos de forma direta;
- Não cria etapas extras durante a execução;

Isso reduz a sobrecarga interna e o tempo total de execução.

Dessa forma, a análise mostra que a diferença entre as curvas do gráfico está ligada principalmente à forma como cada versão é executada pelo hardware, e não à quantidade total de movimentos necessários.

—Análise dos resultados

Ambos possuem a mesma complexidade assintótica $O(2^n)$

No entanto, a versão iterativa apresentou desempenho consistentemente superior em todos os testes realizados.

Portanto, o estudo evidencia que:

- A complexidade assintótica define o comportamento de crescimento do algoritmo;
- A forma de implementação influencia diretamente o tempo prático de execução.

Isso reforça a importância de analisar tanto a teoria quanto a prática no estudo de algoritmos.

Obrigado por assistir

Refêrencias

Torres de Hanoi (artigo) | Algoritmos. Disponível em: <https://pt.khanacademy.org/computing/computer-science/algorithms/towers-of-hanoi/a/towers-of-hanoi>.

PROFESSOR DOUGLAS MAIOLI. Torre de Hanói - Aula 01 - Regras, Como resolver e Fórmula da quantidade de movimentos. Disponível em: <https://www.youtube.com/watch?v=Q2BooYpqS6g>. Acesso em: 23 abr. 2026.

PROFESSOR DOUGLAS MAIOLI. Torre de Hanói - Aula 02 - Demonstração por indução da fórmula da quantidade de movimentos. Disponível em: <https://www.youtube.com/watch?v=ieaHP5_0QiA>. Acesso em: 23 abr. 2026.

CORMEN, T. H. et al. Introduction to Algorithms, third edition. [s.l.] MIT Press, 2009.